Institute for Thermal Turbomachinery
Prof. Dr.-Ing. H.-J. Bauer, Ord.

Analysis of Feature Importance in a Neural Network for Predicting the Remaining Useful Life of Aircraft Engines

Master's Thesis
by
Callum Dew, B.Sc.

Supervisor: **Joel Arweiler, M.Sc.**

April 2026

Karlsruher Institut für Technologie

Ich versichere, die Arbeit selbständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet zu haben. Die wörtlich oder inhaltlich übernommenen Stellen sind als solche kenntlich gemacht. Die Satzung des Karlsruher Instituts für Technologie (KIT) zur Sicherung guter wissenschaftlicher Praxis wurde von mir in der aktuellen Fassung beachtet.

Karlsruhe, April 4, 2026

Contents

List of Symbols iii

List of Figures iv

List of Tables v

1 Introduction 1

2 Related Work 2
- 2.1 Data-Driven Remaining Useful Life Prediction . . . 2
- 2.2 Explaibale Artificial Intelligence in Prognostics . . . 3
- 2.3 Research Gap and Thesis Positioning . . . 4

3 Fundamentals 6
- 3.1 Model-Agnostic Methods . . . 6
  - 3.1.1 Individual Conditional Expectation . . . 6
  - 3.1.2 Feature Ablation . . . 7
  - 3.1.3 Shapley Additive Explanations . . . 7
  - 3.1.4 Local Interpretable Model-Agnostic Explanations . . . 8
  - 3.1.5 Counterfactual Explanations . . . 8
  - 3.1.6 Permutation Feature Importance . . . 9
  - 3.1.7 Accumulated Local Effects . . . 9
- 3.2 Model-Specific Methods . . . 10
  - 3.2.1 Integrated Gradients . . . 10

4 Methodology 12
- 4.1 Dataset and Data Preparation . . . 12
  - 4.1.1 Features set / sensors used . . . 12
  - 4.1.2 Target definition (RUL label) . . . 13
- 4.2 Model Setup . . . 14
- 4.3 Training Procedure . . . 14
  - 4.3.1 Model Configurations . . . 15
- 4.4 Implementation of Feature Importance Methods . . . 16
- 4.5 Unified Feature Importance Representation . . . 19

5 Results 20
- 5.1 Verification of FI algorithms . . . 20
- 5.2 Individual Method Results . . . 21
  - 5.2.1 ICE Curves . . . 22
  - 5.2.2 SHAP Analysis . . . 22
- 5.3 Comparative Feature Importance Across Methods . . . 23
  - 5.3.1 Single-Dataset Models . . . 23

5.3.2 Multi-Dataset Models 24
5.4 Patterns Across Configuration 26
5.5 Feature Importance Results per Method 26
5.6 Comparison 26
5.7 Discussion 27

6 Conclusion and Outlook **28**
6.1 Summary of Contributions 28
6.2 Answering the Research Questions 28
6.3 Outlook 28

References **29**

Appendix **32**
A.1 Individual Results 32
A.2 Comparative Results 32

List of Symbols

Symbol	Unit	Description
Latin symbols		
F_{xy}		Faktor xy
H	m	Höhe
c_p	$\mathrm{J/(kg\,K)}$	Spezifische Wärmekapazität
Greek symbols		
κ		Isentropenexponent
λ	$\mathrm{W/(m\,K)}$	Wärmeleitfähigkeit
ε	$\mathrm{m^2/s^3}$	Dissipationsrate
Similarity parameters		
Bi		Biotzahl
Nu		Nußeltzahl
Pr		Prandtlzahl
Re		Reynoldszahl
Indices		
r		r-Koordinate
x		x-Koordinate
y		y-Koordinate
Abbreviations		
CFD		Computational Fluid Dynamics
FEM		Finite-Elemente-Methode

List of Figures

4.1 Structure of the input tensor for a single sliding window sample, consisting of 13 sensor channels over 70 consecutive time steps. Each slice in the depth direction represents one engine unit. 13

4.2 Architecture of the 1D convolutional neural network. The red window illustrates the receptive field of the convolutional kernel ($k = 5$). Green: feature extractor, blue: label predictor. 15

5.1 Verification of feature importance implementations on a linear model with known weights. Bars show estimated weight; black markers indicate the true weights. . 20

5.2 L1-normalised sensor importance across all eight methods for the single-FD001 model. Sensors are ordered by decreasing mean importance. 23

5.3 L1-normalised sensor importance across all eight methods for the single-FD002 model. Sensors are ordered by decreasing mean importance. 24

5.4 L1-normalised sensor importance across all eight methods for the pairwise-FD001-FD002 model. Sensors are ordered by decreasing mean importance. . . 25

5.5 L1-normalised sensor importance across all eight methods for the model trained on all subsets. Sensors are ordered by decreasing mean importance. 25

List of Tables

2.1 Overview of the C-MAPSS dataset subsets Saxena et al. (2008). 2

4.1 Sensor features retained after non-informative sensor removal. 12
4.2 Overview of all model configurations used in this work. 16

1 Introduction

The aviation industry is one of the most safety-critical and technologically sophisticated fields in the world. Ensuring the reliability and availability of aircraft is essential not only for passenger safety but also for the economic efficiency of airlines and engine manufacturers. As engines operate under extreme mechanical and thermal conditions, their degradation processes must be continuously monitored and understood. In recent years, predictive maintenance has become a central concept in aviation, aiming to anticipate component failures before they occur and to enable maintenance actions precisely when needed.

A key element of predictive maintenance is the estimation of the Remaining Useful Life (RUL) of engine components. RUL predictions allow operators to plan maintenance more effectively, reduce unscheduled downtime, and avoid unnecessary part replacements. However, obtaining accurate RUL predictions is challenging. The underlying degradation mechanisms are highly nonlinear and influenced by a large number of operational parameters. Traditional modelling techniques often struggle to capture this complexity.

With the rise of large sensor datasets and computational power, machine learning and deep learning methods have become increasingly important in RUL estimation. Neural networks, in particular, are capable of modelling complex relationships in engine data and have demonstrated strong predictive performance. However, these models introduce a new challenge: despite their accuracy, they often behave as black boxes, offering little insight into how individual features contribute to the prediction. In a safety-critical environment such as aviation, this lack of transparency poses a significant limitation. Understanding why a model arrives at a certain prediction is essential for trust, certification, and operational decision-making.

This challenge has led to the emergence of Explainable Artificial Intelligence (XAI). XAI methods aim to make complex AI systems more interpretable by providing insights into which input feature influence a model's output and to what extent. One of the most widely used approaches within XAI is the analysis of feature importance. Feature importance methods allow engineers and researchers to understand which parameters are most relevant for RUL estimation and how different models may focus on different aspects of the data.

2 Related Work

2.1 Data-Driven Remaining Useful Life Prediction

The dataset used throughout this tesis is C-MAPSS (Commercial Modular Aero-Propulsion System Simulation), published by Saxena et al. (2008) for the PHM 2008 data challenge. It contains four subsets of simulated turbofan engine run-to-failure trajectories, FD001 through FD004, each with a different combination of operating conditions and fault modes. Every engine unit records 21 sensor measurements and three operatinoal settings per cycle. Degradation is injected into the high-pressure compresso(HPC), the fan, or both, depending on the subset. 2.1 gives the breakdown.

Subset	Training Engines	Test Engines	Operating Conditions	Fault Modes
FD001	100	100	1 (Sea Level)	1 (HPC Deg)
FD002	260	259	6	1 (HPC Deg)
FD003	100	100	1 (Sea Level)	2 (HPC + Fan Deg)
FD004	248	249	6	2 (HPC + Fan Deg)

Table 2.1: Overview of the C-MAPSS dataset subsets Saxena et al. (2008).

FD001 is the simples case, one operating condition, one fault mode. FD004, with six operating conditioins and two fault modes, is the hardest. The multi-condition subsets (FD002 and FD004) contain roughly two and a half times as many engines, reflecting the additional variablility. C-MAPSS has appeared in well over a hundred publications since its release and remains the go-to benchmakr, mostly because nothing else offers the same scale with controlled degradation across multiple regimes.

Nearly all C-MAPSS studies adopt a piecewise linear(PwL) labelling scheme for the RUL target. Heimes (2008) showed with a multilayer perceptron that prediction improve when the early, healthy portion of each trajectory is assigned a constant RUL rather than a linearly decreasing one. The reasoning is straightforward: sensors show no measurable degradation in the early cycles, so there is nothing in the data to distinguish an engine at cycle 10 from one at cycle 50. Assigning a constant cap avoids forcing the model to learn distinctions that the sensor readings do ot support. Heimes set this cap at 130 cycles. Later work has used values between 115 and 130; this thesis uses 125, which is among the most common choices. The value is heruistic, and it matters, particularly on FD002 and FD004, where trajectory lengths vary more.

The first generation of data-driven RUL methods relied on classical machine learning. Wang et al. (2008) won the PHM 2008 challenge with a similarity-based algorithm. Support vector machines, random forests and ensemble approaches followed. Ramasso and Saxena (2014) reviewed this body of work and noted a recurring dependence on hand-crafted health indicators, features built from domain knowledge about which sensors degrade and how. This is effective, but hard to scale.

Deep learning changed that. Sateesh Babu et al. (2016) were among the first to apply a deep Convolutional Neural Network (CNN) to RUL prediction, treating the problem as a regression over multivariate time-series inputs. Li et al. (2018) extended this direction with a deeper architecture and a systematic sliding-window sample preparation scheme, feeding raw sensor data directly into the network without manual feature engineering. Their model outperformed most prior methods on C-MAPSS and became one of the most cited CNN-based approaches in the prognostics literature. Long Short-Term Memory(LSTM)-based approaches appeared around the same time (Wu et al., 2018, Zheng et al., 2017), exploiting the sequential structure of the degradation data. Hybrid architectures combining CNN feature extraction with LSTM temporal modelling soon followed (e.g., Zhao et al. (2017)), and attention mechanisms were layered on top, for instance, Li et al. (2022) added a convolutional block attention module (CBAM) to a CNN-LSTM on C-MAPSS, allowing the model to weight sensor channels and time steps selectively. Transformer-based models and domain adaptation techniques are the most recent additions, through the core task, predicting a scalar RUL from a sensor window, has stayed the same throughout.

The model in this thesis is a 1D-CNN: convolutional layers for feature extraction, fully connected layers for regression. No recurrence, no attention, no gating. That structural simplicity matters here, because the focus is on interpretability. Fewer internal mechanism mean fewer confounding factors when attributing prediction back to individual input features.

2.2 Explaibale Artificial Intelligence in Prognostics

Strong RUL predictions are not enough on their own. In aviation maintenance, where sensor readings feed into decision about grounding aircraft or scheduling inspections, engineers need to understand what a model is responding to. A prediction driven by physically meaningful degradation signals is useful; one driven by spurious correlations in the training data is dangerous. This tension between predictive accuracy and interpretaility runs through the broader machine learning literature Rudin (2019), but it is especially acute in satey-critical prognostics, where a wrong explanatino attached to a correct prediction can mislead maintenance decisions just as badly as an incorrect prediction.

Applying XAI to prognostic models is still uncommon. Serradilla et al. (2020) applied Local Interpretable Model-agnostic Explanations (LIME; Ribeiro et al. (2016)) permutation-based feature importance to a random forest-based RUL estimator for industrial bushings, using the explanations alongside domain knowledge to guide model development. Theier work demonstrated the principle, that XAI tools can do more than just post-hoc reporting, but was limited to a relatively simple model on non-aviation data.

On C-MAPSS specifically, Hong et al. (2020) were among the first, using SHapley Additive exPlanations (SHAP; (Lundberg and Lee, 2017a)) on a deep CNN-Bidirectional LSTM. They first reduced the sensor set through correlation analysis and regularisation, then ran SHAP on the reduced inputs. The resulting force plots broke down individual predictions by sensor contribution. Useful as a proof of concept, but the analysis only covered one explanation method,

and because sensors had already been filtered before the SHAP step, the attributions described a pre-selected feature space rathen than what the model would have learned from the full suite.

Baptista et al. (2022) compared SHAP attributions against traditional prognostic metrics, monotonicity, trendability, prognostability, that are commonlgy used to assess sensor quality before modelling. The two did not always agree: sensor scoring well on prognostic metrics were not necessarily the ones the trained model weighted most heavily. That disconnect says something about the gap between a priori feature assessment and what a network actually extracts from the data during training.

Baptista et al. (2024) then turned to LIME (Ribeiro et al., 2016), applying it to a Gated Recurrent Unit(GRU) Network on C-MAPSS. LIME recovered both global and local feature importance, but the local explanation quality was uneven. Late in the degradation trajectory, where the signal is strong and clear, surrogate fidelity was reasonable. Early in life, with weak degradation signatures and noisy readings, it dropped off sharply, a known limitation of perturbation-based methods on time-series data, but worth confirming empirically.

Alomari et al. (2023) took SHAP in a different direction,, using it not just for post-hoc explanation but as part of the feature selection pipeline before training. Similar workflows have since appeared, coupling SHAP-based selection with CNN-LSTM or Temporal Convolutional Networks (TCN)-LSTM architectures. Gawde et al. (2024) applied four methods - SHAP, LIME, Partiall Dependence Plots (PDP), and Individual Conditional Expectation (ICE; Goldstein et al. (2015)) - to rotating machinery health monitoring, demonstrating a multi-method comparison, through on a different domain and with classical ML models rather than deep networks.

The methodological range across the C-MAPSS prognostics literature remains narrow. SHAP dominates. LIME appears occasionally. Gradient-based attribution methods, marginal-effect methods, and counterfactual approaches, all well estabileshed in the broader XAI field (see 3), are absent from C-MAPSS RUL work almost entirely. The few multi-method studies that do exist, like Gawde et al. (2024), apply to different equipment and simpler models. On deep networks trained for turbofan prognostics, no comparative evaluation has been carried out.

2.3 Research Gap and Thesis Positioning

The literature reviewed above reveals two things. First, deep learning models for RUL prediction on C-MAPSS have matured rapidly, from basic CNNs and LSTMs to attention-augmented hybrids and Transformers, but the question of what these models actually learn from the sensor data has received far less attention. Second, the small body of work that does address interpretability in this domain relies almost exlusively on SHAP, occasionally LIME, applied in isolation to a single architecture.

This is a problem, because different explanation methods operate on different principles and can produce differnt, sometimes contradictory, results for the same model and input. A gradient-based method like Integrated Gradients (Sundararajan et al., 2017) traces attributions through the network's own parameters. A perturbation-based method like Permutation Feature Impor-

tance (Breiman, 2001) observes how prediction change when inputs are shuffled. A marginal-effect method like Accumulated Local Effects (ALE; Apley and Zhu (2019)) characterise the prediction surface shifts as a single feature varies. Relying on one method alone risks mistaking that method's particular perspective for a complete picture of the model's behaviour.

This thesis addresses the gap by applying eight feature importance methods (see 3) to a 1D CNN trained on a variety of subset combinations of all four C-MAPSS subsets. The contribution is not just an expanded set of explanations, but a systematic comparison, identifying where methods agree on which sensor drive the model's prediction, where they diverge and what effect the different subset combinations have on the model.

3 Fundamentals

This chapter covers the theoretical foundations of the feature importance methods used in this thesis, including formal definitions, mathematical formulations, and key properties of each technique. The methods are divided into model-agnostic and model-specific approaches; application-specific details are addressed in Chapter 4.

3.1 Model-Agnostic Methods

Model-agnostic feature importance methods operate independently of the underlying model structure and require only the ability to query the model with input data and observe the resulting output. Let

$$f : \mathbb{R}^d \to \mathbb{R} \tag{3.1}$$

be a trained predictive model mapping an input vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$ to a scalar output such as a predicted RUL. This definition of f and $\mathbf{x}$ is retained throughout this section. Without access to the model internals, these methods analyse how the output changes when inputs are perturbed, fixed, permuted, or replaced, yielding either global or local explanations depending on whether effects are evaluated across the full dataset or at the level of individual samples.

3.1.1 Individual Conditional Expectation

Individual Conditional Expectation (ICE) visualises how a model's prediction changes as a single feature varies while all other features are held fixed at their observed values (Goldstein et al., 2015). For a sample $\mathbf{x}^{(i)} = (x_1^{(i)}, \ldots, x_d^{(i)})$, where superscript (i) identifies the sample and subscript j the feature of interest, the ICE curve is obtained by substituting a grid of values z into position j:

$$\mathrm{ICE}_j\left(\mathbf{x}^{(i)}, z\right) = f\left(x_1^{(i)}, \ldots, x_{j-1}^{(i)}, z, x_{j+1}^{(i)}, \ldots, x_d^{(i)}\right). \tag{3.2}$$

Each evaluation replaces x_j with z while all other features remain fixed, producing a prediction curve over the range of z for that instance. Averaging across all N samples gives the Partial Dependence Plot (PDP),

$$\mathrm{PDP}_j(z) = \frac{1}{N} \sum_{i=1}^{N} \mathrm{ICE}_j\left(\mathbf{x}^{(i)}, z\right), \tag{3.3}$$

which describes the average marginal effect of x_j across the data distribution. Where individual ICE curves diverge substantially, this signals feature interactions or non-linear effects that the PDP would obscure through averaging. In this thesis, a scalar importance score is derived from ICE by computing the mean variance of individual ICE curves across all instances,

$$\text{ICE-FI}_j = \frac{1}{M} \sum_{i=1}^{M} \mathrm{Var}_z\left[\mathrm{ICE}_j\left(\mathbf{x}^{(i)}, z\right)\right], \tag{3.4}$$

which measures how much each instance's prediction varies as feature j is swept across its range, averaged over all M instances (see Chapter 4). This captures instance-level sensitivity

more robustly than the range of the PDP, since it accounts for the full spread of the curve across all grid points rather than relying on the endpoints alone.

3.1.2 Feature Ablation

Feature Ablation quantifies feature importance by measuring how much the model's prediction changes when a feature is replaced by a neutral reference value (Zeiler and Fergus, 2014). If removing a feature's information causes a large shift in the output, that feature matters.

To evaluate feature x_j, a modified input $\tilde{\mathbf{x}}_j$ is constructed by replacing x_j with a reference value x_j^{ref}, such as the feature mean or zero, while all other features remain unchanged. The local attribution is

$$\phi_j(\mathbf{x}) = f(\mathbf{x}) - f(\tilde{\mathbf{x}}_j). \tag{3.5}$$

A large absolute value of $\phi_j(\mathbf{x})$ indicates strong local influence of feature j. In multivariate time-series data, entire sensor channels or time windows can be ablated jointly, enabling interpretation at a higher semantic level. Aggregating over the dataset gives a global importance score:

$$\phi_j^{\text{FA}} = \frac{1}{N} \sum_{i=1}^{N} \left| \phi_j\left(\mathbf{x}^{(i)}\right) \right|. \tag{3.6}$$

Attributions depend strongly on the choice of x_j^{ref}, and replacing features independently can generate inputs that lie far outside the training distribution when features are statistically dependent (Molnar, 2022).

3.1.3 Shapley Additive Explanations

SHAP (SHapley Additive exPlanations) attributes a model prediction to its input features using cooperative game theory (Lundberg and Lee, 2017b, Shapley, 1953). Each feature is treated as a player, and its attribution is its average marginal contribution to the prediction across all possible feature subsets.

A value function $v(S) \in \mathbb{R}$, defined for each subset $S \subseteq F = \{1, \ldots, d\}$, quantifies the model output when only the features in S are active, with absent features replaced by a reference value or marginalised out. The Shapley value for feature j is

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(d - |S| - 1)!}{d!} \left(v(S \cup \{j\}) - v(S) \right), \tag{3.7}$$

where the factorial weight averages the marginal contribution $v(S \cup \{j\}) - v(S)$ over all coalition sizes and orderings (Shapley, 1953). The attributions form an additive decomposition of the prediction:

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{d} \phi_j, \tag{3.8}$$

where $\phi_0 = \mathbb{E}[f(X)]$ is a baseline value under a background distribution (Lundberg and Lee, 2017b). Four axioms uniquely characterise the Shapley values: *efficiency* ensures attributions sum to the explained quantity; *symmetry* gives equal attribution to features contributing identically to all coalitions; the *dummy property* assigns zero to features that never change the value function; and *linearity* ensures attributions combine linearly for linear combinations of models.

Two common formulations of $v(S)$ exist. Setting $v(S) = \mathbb{E}[f(X) \mid X_S = x_S]$ respects feature dependencies but requires estimating high-dimensional conditional distributions. Setting $v(S) = \mathbb{E}[f(\mathbf{x}_S, X_{\bar{S}})]$ draws absent features from a background distribution independently, which is computationally simpler but may evaluate the model at unlikely input combinations. Both formulations can yield different attributions when features are correlated (Aas et al., 2021). Exact computation requires evaluating all 2^d coalitions, which quickly becomes intractable; in practice, Shapley values are approximated via sampling-based estimation (Štrumbelj and Kononenko, 2014) or weighted regression (Lundberg and Lee, 2017b), at the cost of some variance in the estimates.

3.1.4 Local Interpretable Model-Agnostic Explanations

LIME (Local Interpretable Model-Agnostic Explanations) explains an individual prediction by fitting a simple interpretable surrogate to the original model in the neighbourhood of a specific input instance (Ribeiro et al., 2016). Even globally complex models often behave approximately linearly in a small enough region, which is the premise LIME exploits.

Given an instance $\mathbf{x} \in \mathbb{R}^d$, LIME samples perturbed inputs $\{\mathbf{z}_k\}_{k=1}^K$ near $\mathbf{x}$ and evaluates each to obtain $f(\mathbf{z}_k)$. A locality kernel $\pi_{\mathbf{x}}(\mathbf{z}_k)$ down-weights samples distant from $\mathbf{x}$, focusing the surrogate on the immediate neighbourhood of the instance. A sparse linear model g is then fitted by solving

$$\arg\min_{g \in \mathscr{G}} \mathscr{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g), \tag{3.9}$$

where $\mathscr{L}$ measures how closely g approximates f under the locality weighting and $\Omega(g)$ penalises complexity (Ribeiro et al., 2016). For a linear surrogate,

$$g(\mathbf{z}) = w_0 + \sum_{j=1}^{d} w_j z_j, \tag{3.10}$$

the fitted coefficients w_j are the local feature importance scores, directly indicating each feature's contribution at $\mathbf{x}$. Explanations can vary across runs due to the stochastic nature of the perturbation sampling.

3.1.5 Counterfactual Explanations

Counterfactual explanations take a different perspective from attribution-based methods: instead of scoring each feature, they ask what the smallest change to the input would be to shift

the model's prediction to a desired target. Features that must change most are considered the most influential (Wachter et al., 2018).

For an instance $\mathbf{x} \in \mathbb{R}^d$ and a target y', a counterfactual $\mathbf{x}^{\text{cf}}$ is sought satisfying $f(\mathbf{x}^{\text{cf}}) = y'$ while minimising the distance $D(\mathbf{x}, \mathbf{x}^{\text{cf}})$. In practice this constrained problem is relaxed to

$$\mathbf{x}^{\text{cf}} = \arg\min_{\mathbf{z}} \lambda \cdot \mathscr{L}(f(\mathbf{z}), y') + D(\mathbf{z}, \mathbf{x}), \tag{3.11}$$

where $\mathscr{L}$ penalises deviation from y' and $\lambda > 0$ controls the trade-off between prediction fidelity and closeness to the original input (Wachter et al., 2018). D is typically an L_1 or L_2 norm. Additional constraints can be imposed to ensure plausibility, sparsity, or feasibility (Verma et al., 2022, **?**), and causal structure can be incorporated to respect feature dependencies (Kusner et al., 2018). The optimisation is generally non-convex, and multiple valid counterfactuals may exist for any given instance.

3.1.6 Permutation Feature Importance

Permutation Feature Importance (PFI) quantifies each feature's contribution by measuring the drop in predictive performance when the statistical link between that feature and the target is broken (Breiman, 2001, Fisher et al., 2019). Shuffling a feature's values across samples removes any information it carries about the target while leaving its marginal distribution unchanged, any resulting performance loss is then attributable to the loss of that feature's signal.

Let $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ be the evaluation dataset and $\mathscr{L}(f)$ a performance metric such as mean squared error. For feature j, a permuted dataset $\tilde{\mathbf{x}}^{(j)}$ is constructed by randomly shuffling x_j across all samples while leaving all other features unchanged. The permutation importance is

$$\text{PFI}_j = \mathscr{L}\Big(f(\tilde{\mathbf{x}}^{(j)})\Big) - \mathscr{L}(f(\mathbf{x})), \tag{3.12}$$

where a larger value indicates greater model reliance on feature j for error-based metrics. PFI measures predictive dependence, not causal effect. When features are correlated, permuting one independently of the others creates combinations unlikely to appear in training, which can inflate or distort the estimates (Fisher et al., 2019).

3.1.7 Accumulated Local Effects

Accumulated Local Effects (ALE) estimates the average influence of a feature on model predictions in a way that accounts for feature correlations (Apley and Zhu, 2019). The standard alternative, PDP, averages predictions over the marginal distribution of all other features, which can produce evaluations at unrealistic input combinations when features are dependent. ALE addresses this by computing prediction differences within narrow local intervals, conditioning only on samples that actually fall in that region.

The domain of feature x_j is partitioned into K disjoint intervals at grid points $z_0 < z_1 < \cdots < z_K$, chosen as empirical quantiles to ensure sufficient data in each bin. For each sample $\mathbf{x}^{(i)}$ in

interval $[z_{k-1}, z_k]$, the local prediction difference is

$$\Delta_{j,k}^{(i)} = f\left(x_1^{(i)}, \ldots, z_k, \ldots, x_d^{(i)}\right) - f\left(x_1^{(i)}, \ldots, z_{k-1}, \ldots, x_d^{(i)}\right), \tag{3.13}$$

with all other features held at their observed values. The average within interval k,

$$\hat{\Delta}_{j,k} = \frac{1}{N_k} \sum_{i: x_j^{(i)} \in [z_{k-1}, z_k]} \Delta_{j,k}^{(i)}, \tag{3.14}$$

is then accumulated to give the ALE function at point z:

$$\mathrm{ALE}_j(z) = \sum_{k: z_k \leq z} \hat{\Delta}_{j,k}. \tag{3.15}$$

The function is centred so that

$$\int \mathrm{ALE}_j(z)\, dP_{X_j}(z) = 0 \tag{3.16}$$

meaning it reflects deviations from the average prediction rather than absolute values. Unlike ICE, which traces the full prediction curve per instance, ALE describes the average local rate of change across bins, a global perspective. The PDP formulation $\mathbb{E}[f(x_j, X_{\setminus j})]$, where $X_{\setminus j}$ denotes all features except j, averages over the marginal distribution, which can place the model in regions of the input space that are practically unreachable; ALE is specifically designed to avoid this. Instability in sparse regions of the feature space remains a limitation.

3.2 Model-Specific Methods

Model-specific feature importance methods make use of quantities internal to the model, gradients, activations, or parameter values and are therefore tied to a particular model class. Unlike model-agnostic methods, which work purely from input-output pairs, they require access to the model internals, restricting their applicability to differentiable architectures such as neural networks. Within that scope, they tend to be more computationally efficient and come with stronger theoretical guarantees.

3.2.1 Integrated Gradients

Integrated Gradients (IG) is a gradient-based attribution method for differentiable models (Sundararajan et al., 2017). Plain gradient saliency evaluates only the local gradient at the input, which can be uninformative when the model saturates. IG instead accumulates gradients along a straight-line path from a reference baseline to the instance being explained, building a more complete account of each feature's contribution.

Let $\mathbf{x} \in \mathbb{R}^d$ be the input and $\mathbf{x}' \in \mathbb{R}^d$ a baseline representing an uninformative reference state, for example, an all-zeros vector. The path between them is the vector-valued interpolation

$$\gamma(\alpha) = \mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}') \in \mathbb{R}^d, \quad \alpha \in [0, 1], \tag{3.17}$$

where α is a scalar interpolation parameter and $\gamma(\alpha)$ the corresponding input vector. The attribution for feature j is the path integral of the partial derivative of f with respect to x_j, scaled by the displacement of that feature between baseline and input:

$$\phi_j(\mathbf{x}) = (x_j - x'_j) \int_0^1 \frac{\partial f(\gamma(\alpha))}{\partial x_j} \, d\alpha. \tag{3.18}$$

In practice the integral is approximated by a Riemann sum with m steps:

$$\phi_j(\mathbf{x}) \approx (x_j - x'_j) \frac{1}{m} \sum_{k=1}^{m} \frac{\partial f(\gamma(\alpha_k))}{\partial x_j}, \quad \alpha_k = \frac{k}{m}. \tag{3.19}$$

IG satisfies the *completeness* property: attributions sum exactly to the difference in model output between input and baseline,

$$\sum_{j=1}^{d} \phi_j(\mathbf{x}) = f(\mathbf{x}) - f(\mathbf{x}'), \tag{3.20}$$

giving a fully additive decomposition of the prediction difference. Two further axioms are satisfied: *sensitivity*, a feature responsible for a prediction difference always receives non-zero attribution, and *implementation invariance*, functionally equivalent models produce identical attributions (Sundararajan et al., 2017). Attributions can differ substantially with different baseline choices, and no universally appropriate baseline exists. Global feature importance is obtained by aggregating local attributions across samples.

4 Methodology

This chapter describes the experimental setup used to analyse feature importance in neural network-based RUL prediction. It covers the dataset and preprocessing pipielin, the model arcchitecture and its three configurations, the training procedure, and the implementation of feature importance methods. The models themselves are not the primary contribution of this thesis; they serve as controlled predictive systems under which feature importance behaviour can be systematically studied across different input configurations.

4.1 Dataset and Data Preparation

4.1.1 Features set / sensors used

The raw input space of each CMAPSS sub-dataset consitst of three operational settings and 21 sensor measurements per time step. Eight sensors: s1, s5, s6, s10, s14, s16, s18, and s19, were identified as non-informative due to near-constant values across all trajectories and were excluded from the analysis. The remaining 13 sensor channels, listed in 4.1, form the feature set used throughout this work.

Table 4.1: Sensor features retained after non-informative sensor removal.

Senso ID	Physical Description
s2	LPC outlet temperature
s3	HPC outlet temperature
s4	LPT outlet temperature
s7	HPC outlet pressure
s8	Fan speed
s9	Core speed
s11	HPC outlet static pressure
s12	Ratio to Fuel flow to Ps30
s13	Corrected fan speed
s15	Bypass ratio
s17	Bleed enthalpy
s20	HPT cool air flow
s21	LPT cool air flow

Each sensor channel is scaled indepentently to the range [0, 1] using a MinMax scaler. Because sensosr readings vary systematically with ooperating condition, scaling is applied separately within each of the distinct operating regimes present in the data. The scaler is fitted exclusively on the training set.

Input sequences are constructed using a sliding window of length $T = 70$ time steps and stride 1, yielding tensors of shape (N x 30 x 70), where N is the number of sequences. The label

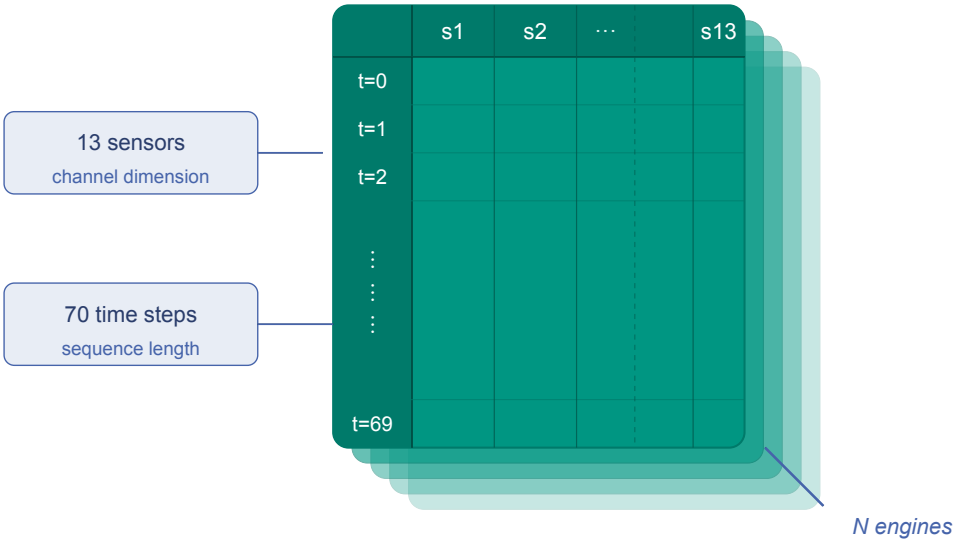

Figure 4.1: Structure of the input tensor for a single sliding window sample, consisting of 13 sensor channels over 70 consecutive time steps. Each slice in the depth direction represents one engine unit.

associated with each window is the RUL value at the final time step of that window. The resulting input strcture is illustrated in Figure 4.1.

4.1.2 Target definition (RUL label)

The raw RUL for each engine trajectory is computed as the difference between the end-of-life cycle t_{EOL} and the current cycle t:

$$\text{RUL}_{\text{raw}}(t) = t_{\text{EOL}} - t \tag{4.1}$$

A purely linear RUL label is known to dominate model training with the healthy regime, where no meaningful degradation signal is present. To address this, a piecewise linear transformatino is applied using a maximum RUL threshold $R_c = 125$ cycles. Early-life cycles, where the tru RUL exceeds R_c, are assigned a constant label of R_c, under the assumptioin that the engine shows no detectable degradation in this regime. Only once the trajectory enters the degradation region does the label decrease linearly toward zero at failure:

$$\text{RUL}(t) = min(\text{RUL}_{\text{raw}}(t), R_c). \tag{4.2}$$

This transformation prevents the model from being overwhelmed by healthy-state samples and focuses learning on the degradation region, which carries the operationally relevant signal.

Following the piecewise transfromation, RUL labels are normalized to [-1, 1] using a min-max scaling based on the training set:

$$\hat{y} = 2 \cdot \frac{\mathrm{RUL}(t) - \mathrm{RUL}_{\min}}{\mathrm{RUL}_{\max} - \mathrm{RUL}_{\min}} - 1, \tag{4.3}$$

where $\mathrm{RUL}_{\min}$ and $\mathrm{RUL}_{\max} = R_c = 125$ by construction, so that a healthy engine at the cap maps to +1 and an engine at failure maps to -1. The same scaling parameters are applied to the test set without re-fitting.

4.2 Model Setup

All model variants used in this thesis share a common architecture base on a Domain-Adversarial Neural Netwrok (DANN) . The DANN consists of three components: a convolutional feature extractor, a label predictor, and an optional domain classifier connected via a Gradient Reversal Layer (GRL). Since the primary purpose of the models here is to serve as a stable substrate for feature importance analysis rather than to maximise transfer performance, the domain classifier branch is used selectively and the architecture is held fixed across all configurations.

The **Feature Extractor** processes the input tensor of shape $(Nx13xT)$ through four successive one-dimensional convolutional layers. The number of filters follows a halving schedule: 128, 64, 32, and 16 channels respectively, each with kernel size 5 and same-length padding of 2. After each convolutional layer, a LeakyReLu activation with negative slope 0.1 is applied. The temporal dimensions is collapsed using a global average pooling operation (AdaptiveAvgPool1d), which produces a fixed-length representation of 16 values regardless of input sequence length. A final fully connected layer projects this to a feature vector of length 64. Note that the slab height in Figure 4.2 is a visual convention representing the reducing channell count and does not imply a reduction in the temporal dimesnion, which remains 70 throughout.

The **Label Predictor** maps the 64-dimensioinal feature vector to a scalar RUL estimate through two fully connected layers: Linear $(64 \to 32)$ and Linear $(32 \to 1)$, each preceded by a ReLU activation. The output is a single value representing the predicted RUL.

4.3 Training Procedure

All models are trained to minimise the mean squared error (MSE) between the predicted and true normalised RUL values:

$$\mathscr{L} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}^{(i)} - y^{(i)})^2. \tag{4.4}$$

The models are trained with the following subset combinations.

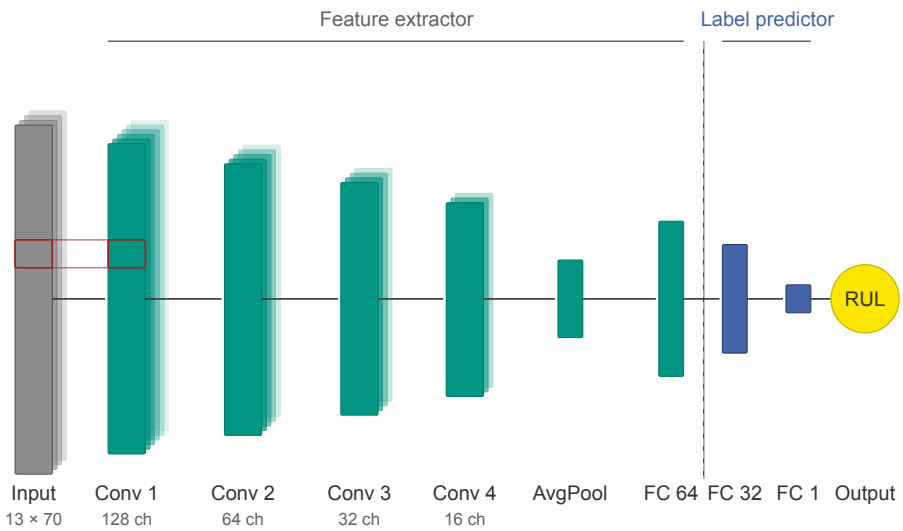

Figure 4.2: Architecture of the 1D convolutional neural network. The red window illustrates the receptive field of the convolutional kernel ($k = 5$). Green: feature extractor, blue: label predictor.

4.3.1 Model Configurations

Three training configurations are studied, differing only in the subset of CMAPSS data used during training. The four single-dataset models each train on one sub-dataset in isolation and serve as the baseline for feature importance behaviour under a single operating condition. The

six pairwise models each train on the concatenated data from two sub-datasets, allowing examination of how feature importance shifts when a model must generalise across two distinct operating regimes simultaneously. The full-dataset model trains on all four sub-datasets combined and represents the most demanding generalisation scenario. Table 4.2 gives an overview of all eleven model configurations.

Table 4.2: Overview of all model configurations used in this work.

Configuration	**Training data**	**No. of models**
Single	FD001	
	FD002	
	FD003	
	FD004	4
Pairwise	FD001 + FD002	
	FD001 + FD003	
	FD001 + FD004	
	FD002 + FD003	
	FD002 + FD004	
	FD003 + FD004	6
Full	FD001 + FD002 + FD003 + FD004	1
Total		**11**

4.4 Implementation of Feature Importance Methods

All eight feature importance methods are applied post-hoc to the trained models, without any modification of the learned weights. A central implementation challenge shared across all methods is the multivariate time-series structure of the input: each sample is a tensor of shape (N x 13 x 70), where the 13 sensor channels and 70 time steps together constitute the model's input spcae. Most standard feature importance implementations assume a tabular input where each column corresponds to an independent scalar feature. Applying these methods to a sequence model therefore requires deliberate desing dicisions about how features are defined, how perturbations are applied and how attributions over the temporal dimesnion are reduced to a per-sensor importance score. Unless stated otherwise, the full temporal extent of a sensor channel is treated as a single unit, perturbations are applied uniformly across all 70 time steps of a given sensor simultaneously and the resulting attributions are aggregated over time to yield a scalar importance score per sensor. This choice reflects the physically motivated view that a sensor's contribution to the RUL prediction should be attributed to the sensor as a whole rather than decomposed into individual time steps, which would result in a 910-dimensional attribution space that is difficutlt to interpret and compare across methods.

Individual Conditional Expectation (ICE): For each of the 13 sensors, a uniform grid of 20 values is constructed between the 5th and 95th empirical percentile of that sensor's observed values across the dataset. The percentile bound are used in place of the global minimum and maximum to avoid evaluatiing the model in the tails of the distribution, where training data is sparse and prediction may be unreliable. For each grid value g and each instance in the dataset, the entire time series of sensor j is replaced by the constant value g, that is, $x_{j,t} = g$ for all $t \in [1, 70]$, while all remaining sensor channels are kept at their observed values. This produces one ICE curve per instance, tracing how the predicted RUL changes as sensor j is swept across its range in isolation. The PDP is then computed as the pointwise mean of all ICE curves. Feature importance is derived as ICE-FI$_j$ (see 3.4), measuring the mean variance of ICE curves across all instances.

Feature Ablation: The implementation follows the grouped ablatioin scheme provided by Captums's *FeatureAblation*, in which all 70 timesteps belonging to a given sensor channel are masked simultaneously as a single group rather than independently. This sensor-grouped formulation is essential for the present setting: ablating individual timesteps in isolation would produce attributions across 910 independent input positions, making cross-method comparison difficult and reducing the physical interpretability of the results. When sensor j is ablated, its entire time series is replaced by the training set mean computed independently for each channel, representing the expected signal in the absence of informative variation. Attributions are defined as the absolute change in prediction relative to the unpertrubed output, and global feature importance is obtained by averaging these absolute differences over all instances in the evaluation set.

Shapley Additive Explanations: To maintain consistency with the grouped ablation scheme described above, the 13 sensor channels are treated as 13 players in the cooperative game rather than operating on the 910-dimensional flattened input. Exact Shapley values are computed by enumerating all $2^{13} = 8192$ possible sensor coalitions and evaluating the model on each, which is computatinoally tractable for this group size. When a sensor group is absent from a coalition, its timesteps are replaced by a mean valued baseline as in the ablation method. For each coalitioin S, the masked input is constructed by replacing the features of all sensors not in S with the baseline and the model is queried to obtain $v(S)$. The Shapley values are then computed from these chached coalition outputs, yielding a per-sensor attribution for each instance in the dataset. A visualistation of the Shapley values is provided in the results chapter and implemented using the *shap* library.

Local Interpretable Model-Agnostic Explanations: The implementation uses Captum's `LIME` with a Lasso linear surrogate as the interpretable model class. Since LIME operates by perturbing the input in an interpretable feature space and fitting a weighted linear regression, the (13 x 70) input tensor is first flattened to a 910-dimensional vector. For each instance to be explained, 4000 perturbed samples are drawn in the neighbourhood of the input, each evaluated by the model. A proximity-weighted Lasso regression($\alpha = 0.01$) is then iftted to approximate the model locally, with the regularisation term enforcing sparsity in the resulting attribution vector. The zero vector is used as the reference baseline. After fitting, the 910-dimesninal weight vector is reshaped back to (13 x 70) and the time dimensioin is averaged to obtain sensor-level

importance scores. The Lasso regularisation parameter α was selected to balance sparsity with attribution stability across instances.

Counterfactual Explanations: Counterfactual explanations are generated using the `DiCE` library, which implements a gradient -based search for minmally modified inputs that shift the model's prediction into a specified target range. As `DiCE` requires a tabular input representation, the (13 x 70) tensor is flattened to a 910-dimensional feature vector before being passed to the `DiCE` data interface, which is initialised using the training set to define plausible feature ranges. For each query instance, the desired output range is specified as a window around the original prediction, so that the counterfactual represents a plasuible alternative trajectory with a meaningfully differen predicted RUL. The distance between the original and counterfactual input is pnealised using an L_1 norm, which encourages sparse modifications, that is, changes concentrated in a small number of features. Feature importance is subseqently derived by computing the magnitued of change per sensor, aggregated across the 70 timesteps, which identifies which sensors were most frequently or substantially modified to achieve the target prediction shift.

Permutation Feature Importance: For each sensor channel, its values are permuted randomly across all N samples in the evaluation set, while all other channels remain at their original values. This destroys the statistical association between that sensor and the target RUL without altering the marginal distribution of the sensor's values, so any resulting drop in predictive performance can be attributed to the loss of that sensor's information. Performance is measured as MSE on the normaliised RUL scale. To account for variablity in the permutation, the procedure is repeated with 5 independent random permutations and the mean and standard deviation of the MSE increase are reported per sensor. The use of 5 repeats provides a sufficiently stable estimate for 13 sensors while keeping the total number of model evaluation manageable.

Accumulated Local Effects: The domain of each sensor is partitioned into 20 bins using quantile-based bin edges, so that each bin contains approximately the same number of samples. Quantile-based binning is preferred over unifrom binning because sensor distributions may be skewed after operating-condition-specific normalisation and equal-frequency bins ensure that local effect estimates are based on suffiecient data throughout the feature range. Bin membership for each sample is determined by the mean value of that sensor's time series, which reduces the (N x L) sensor values to a scalar per sample for assignment purposes. Within each bin, the local prediction difference is computed by replacing the entire time series of senso j with the lower and uper bin boundary values in turm, and recording the resulting change in model output. These local differences are averaged within each bin and then accumulated across bins following 3.16. The accumulated effect curve is centered to zero mean over the empirical distribution of sensor values. A scalar importance scoe is derived as the range of the centered ALE curve, reflecting the total span of predicted RUL variation attributable to that senos under the local conditional distribution.

Integrated Gradients: The implementation uses Captum's `IntegratedGradients` with 100 integration steps and the Gauss–Legendre quadrature approximation, which provides a more accurate estimate of the path integral than a simple Riemann sum for the same number of steps. The `multiply_by_inputs` flag is set to `True`, which scales each gradient by

the difference between the input and the baseline. This ensures that the completeness property holds, i.e. the sum of attributions over all input dimensions equals the difference between the model's prediction at the input and at the baseline. A zero-valued tensor of shape $(1 \times 13 \times 70)$ is used as the baseline, consistent with the choice made for SHAP and Feature Ablation. Since all sensor values lie in $[0,1]$ after MinMax normalisation, a zero baseline represents a meaningful reference state corresponding to the minimum observed sensor value in each channel. Attributions are computed per instance over the full (13×70) input space, absolute values are taken to reflect attribution magnitude regardless of sign, and the temporal dimension is averaged to yield a per-sensor importance score of shape $(13,)$.

4.5 Unified Feature Importance Representation

Because the eight feature importance methods produce attributions in different units and on different scales, a unified representation is required before any cross method comparison can be made. For each method, the raw feature-level attributions are reduced to a single importance score per sensro by aggregating over the temporal dimension where applicable. The resulting scores are then normalised using L_1 normalisation:

$$\mathrm{FI}_{\mathrm{L}_1} = \frac{|\mathrm{FI}_j|}{\sum_{k=1}^{13} |\mathrm{FI}_k|}, \tag{4.5}$$

so that the importance scores for all features sum to one for any given instance or aggregate. This normaliation ensures that the magnitude of attributions is comparable across methods regardless of their original scale, enabling direct visual and quantitative comparison in the results chapter.

5 Results

5.1 Verification of FI algorithms

Before applying the eight feature importance methods to the C-MAPSS models, a controlled verification experiment was carried out to confirm that the implementations behave as expected when ground truth is known. The idea is simple: train a model where the true feature importances are unambiguous, then check whether each method recovers them.

The setup uses a linear function of tn input variables:

$$f(x) = 5x_0 + 3x_1 + 2x_2 + 1.5x_3 + 1X_4 + 0x_5 + 0x_6 + 0x_7 + 0x_8 + 0x_9 \tag{5.1}$$

The first five features carry all the predictive information, with known weights of 5, 3, 2, 1.5 and 1. The remaining five features (x_5 through x_9) have zero weight, they are pure noise. A single-layer linear neural network without activation functions was trained on 4000 samples drawn from a unifrom distribution over [0, 10]. After training, the learned weights matched the true weights to four decimal places, confirming that the model itself was not a source of error.

All eight feature importance methods were then applied to the trained model using the test set. The resulting attributions were scaled to compare true weights with the trained weights, allowing a direct comparison. 5.1 shows the results.

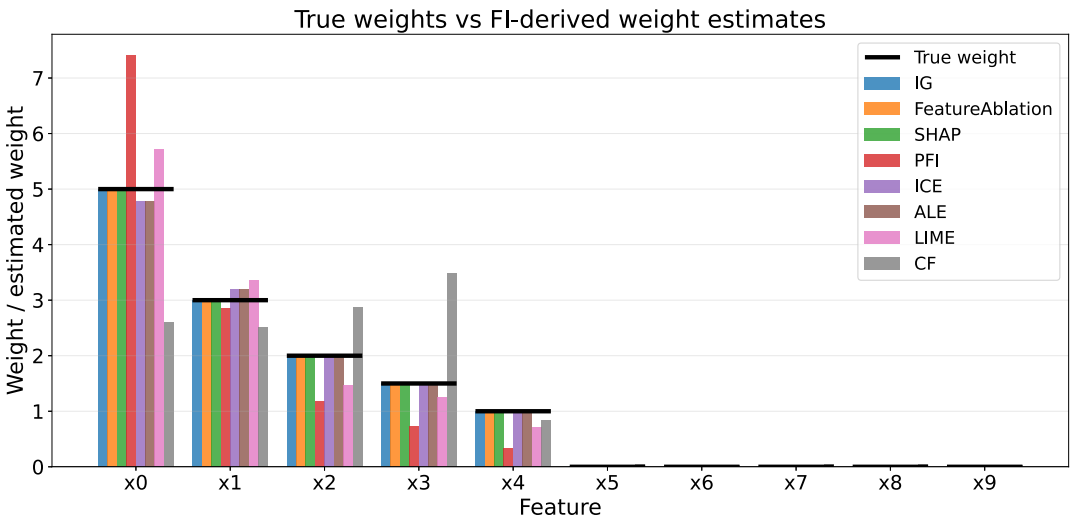

Figure 5.1: Verification of feature importance implementations on a linear model with known weights. Bars show estimated weight; black markers indicate the true weights.

Six of the eight methods, Integrated Gradients, Feature Ablation, SHAP, ALE, ICE and LIME, closely reproduce the true weight distribution. Their bars track the ground truth across all ten

features, correctly assigning near-zero importance to x_5 through x_9 and recovering the correct ranking and approximate magnitudes for x_0 through x_4. Minor deviaitons exist, ICE slightly overestimates the importance of x_0 relative to the other active featrues, but these are within the range expected from sampling variability.

Permutation Feature Importance (PFI) and Counterfactual Explanations (CF) show larger deviations, thugh for different reasons.

PFI overestimates x_0 substantially, roughly 2.5 in estimated weight terms, versus the true value of 5, while compressing the remaining active features downward. The ranking among x_0 through x_4 is preserved and x_5 through x_0 are correctly identified as irrelevant. The skew toward x_0 is a known property of PFI on correlated or hgih-variance features: because PFI measures the increase in prediction error when a feature is permuted, the feature with the largest absolute weight causes the largest error disruption when shuffled, and this effect scale nonlinearly with the weight magnitude under squared-error metrics like MSE. The relative ordering is correct, but the magnitdue distribution does not directly reflect the underlying weight ratios.

Counterfactual Explanations show a different kid of deviation. Rathern than preserving the ranking with distorted magnitudes, CF partially msorders the active features, in the raw weieght estimates, x_3 receives the highest estimated weight and the overall distribution across x_0 through x_4 does not follow the true ordering . This is not unexpected. Counterfactual methods search for the smallest input change that shifts the prediction into a target range. That search depends on the local geometry of the predictino surface and the permitted perturbation ranges, not directly on the magnitude of learned weights. In a linear model the prediction surface is uniformly sloped, which removes the kind of local structure that counterfactual search id designed to exploit, making this an inherently unfavourable test case for the method. On more complex models with nonlinear interactions, such as the 1D CNN used on C-MAPSS, counterfactual explanations may behave more meaningfully.

The key outcome of this verification is that all eight methods correctly distinguish relevant feature from irrelevant ones. The six gradient-based, perturbation-based and marginal-effect methods additionally recover the correct rankin and approximate magnitudes. PFI recovers the ranking but distorts magnitudes. CF struggles with both ranking and magnitudes on this liinear test case, but still separates the two groups cleanly. These results provide confidence that the implementation are functioning correlcy and that observed differences on C-MAPSS reflect genuine properties of the methods rather than implementation errors.

5.2 Individual Method Results

Before comparing the eight methods against each other, it is worth looking at the kind of output they produce individually. This section shows representative ICE curves and SHAP plots, all from the single-FD001 model, the simplest configuration and therefore the easiest to interpret. Complete plots for all model configurations are provided in Appendix A.1

5.2.1 ICE Curves

ICE plots show how the model's RUL prediction changes as a single sensor value is varied across its oberved range, with one curve per test instance. The black line is Partial Dependence (PD) curve, the average across all instances. For s9, the individual ICE curves spread across a range of roughly ± 0.4 in centered RUL, with several raching ± 0.6 or beyond at the extremes of the sensor range. The PD curve shows a mild negative trend, higher s9 values are associated with slightly lower predicted RUL, but the more telling aspect is the spread itself. The wide dispersion of individual curves mean that the effect of s9 on the prediction depends heavily on the values of the other sensor for that particular engine instance. The model is not using s9 in a simple, unifrom way; its influence interacts with the broader sensor context.

Sensor s3 looks entirely different. The PD curve is effectively flat and the individual ICE curves are tightly clustered within ± 0.1. Varying s3 across its full range barely moves the prediction regardless of context. The model has learned to largely ignore this sensor, consistent with the comparative importance rankings in the next section, where s3 appears near the bottom across all methods.

The contrast between these two sensors illustrates what ICE curves are goot ad: they do not just say "this sensor matters more than that one," they show *how* a sensor's effect manifests, whether it is roughly unifrom across instances or highly variable, suggesting interactions with other features.

5.2.2 SHAP Analysis

Figure **??** shows the SHAP beeswarm plot for the single-FD001 model. Each dot represents one test instance; horizontal position indicates the SHAP value (contribution to the prediction) and colour indicates the sensor reading for that instance (red for high, blue for low). Sensor s9 stands out immediately, it has by far the widest spread of SHAP values, ranging from approximately -0.5 to $+0.3$. No other sensor comes close in magnitude. Sensors s4 and s1 form a second tier, with SHAP values reaching roughly ± 0.2. The remaining sensors cluster much more tightly around zero, contributing little to individual predictions. The colour gradient reveals directional effects: for s4, blue dots (low sensor values) tend to produce positive SHAP values and red dots (high values) tend to push negative, meaning that as s4 increases, the modell predicts lower RUL. Sensors further down the ranking, s17 and s3, show very narrow SHAP distributions, confirming that the model assigns them minimal influence on this subset.

Figure **??** shows a SHAP waterfall plot for asingle test instance, breaking down how the model moves from the expected prediction $E[f(X)] = 0.007$ to the instance-specific prediction $f(x) = 0.265$.

Sensor s9 accounts for almost the entire deviation, contributing +0.23, roughly 90% of the shift from the baseline. Sensor s2 adds another +0.12 and the remaining sensors contribute small positive or negative adjustments that partially cancel each other. The waterfall makes visible what the beeswarm shows statistically: onn this model, s9 is the dominant drive of individual predictions and most other sesnros play a supporting or corrective role.

The SHAP plots add directional informatino that the bar chart comparisons in the next section cannot provide. Where the bar charts show *how much* each sesnor matters in aggregate, the beeswarm and waterfall show in *in which direction* and *for which instances*, whether a high sensor reading pushes predicted RUL up or down. This directional detail is useful for sanity-checking the model against physical expectations, a point returnde to in the Discussion.

5.3 Comparative Feature Importance Across Methods

The core of this analysis is the side-by-side comparison of all eight methods on the same model and dataset. Each figure shows L1-normalised sensor importance from every method, with sensors ordererd by decreasing average importance. Four representative configurations are shown: models trained on FD001 alone, FD002 alone, the FD001-FD002 pair, and all four subsets combined. Remaining configurations are included in Appendix A.2.

5.3.1 Single-Dataset Models

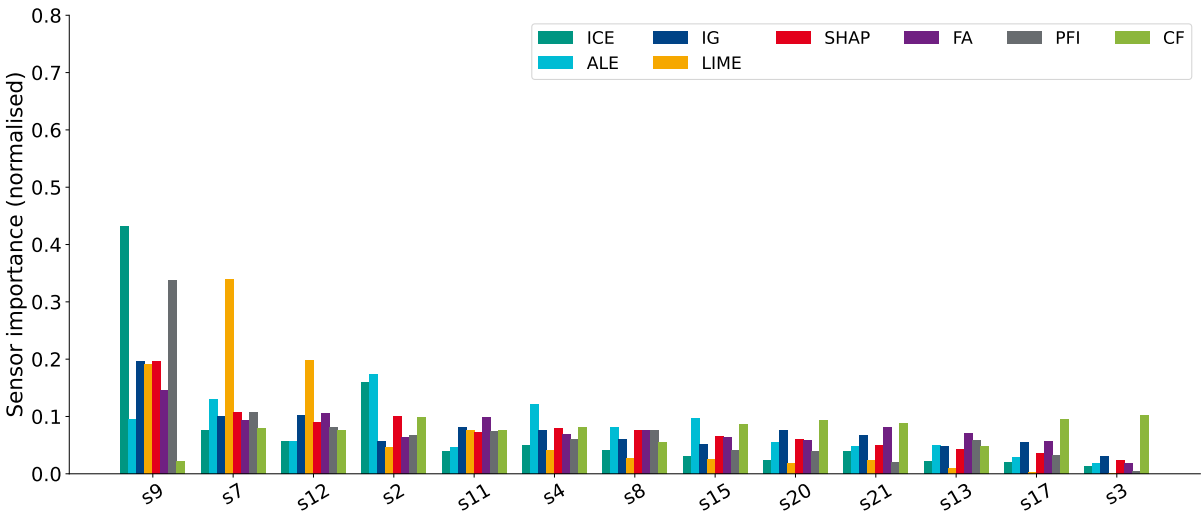

Figure 5.2: L1-normalised sensor importance across all eight methods for the single-FD001 model. Sensors are ordered by decreasing mean importance.

On the single-FD001 model, s9 is the most important sensor according to most methods. ICE assigns it roughly 0.43, PFI around 0.34, and the gradient- and perturbation-based methods (IG, SHAP, FA) converge around 0.19-0.20. The next tier, s7, s12, and s2, is where methods start to diverge. LIME assigns s7 a normalised importance of roughly 0.34, well above what the

other methods give it. ALE places s12 and s4 higher than most alternative. The broad picture is consistent though: s9 leads, a group of four or five sensors follows with moderate importance and the remainder contributes little.

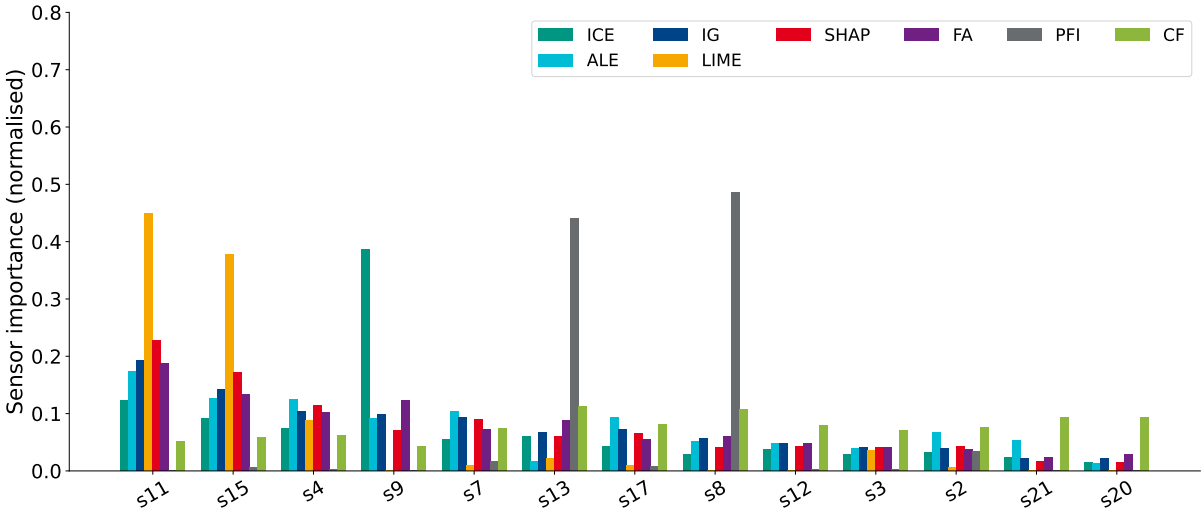

Figure 5.3: L1-normalised sensor importance across all eight methods for the single-FD002 model. Sensors are ordered by decreasing mean importance.

FD002 changes the picture substantially. With sixx operating conditions instead of one, the model faces more complex input distributions and snsor rankings shift accordingly. Sensor s11 emerges as the most important for IG, ALE, SHAP and FA, the four methods that tended to agree most closely on FD001 as well. LIME concentrates even more aggressively, assigning s11 roughly 0.45 and s15 around 0.38, leaving little importance for anything else. PFI produces extreme outlier spikes, s8 receives nearly 0.49 and s13 around 0.44, not reflected by any other method. These PFI spikes are reminiscent of the magnitude distortion observed in the verification experiment and suggest that PFI's MSE-based scoring amplifies the apparent importance of features whose permutation causes large error in isolated instances.

ICE remains erratic in regards to assigning s9 the highest importance at around 0.39 while other methods rank it considerably lower.

5.3.2 Multi-Dataset Models

When the model is trained on FD001 and FD002 together, the rankings stabilise somewhat. Sensors s11 and s15 emerge as the top two across most methods, with s9, s7 ands4 forming a second tier. The pairwise model has seen both single-condition and multi-condition data and the resulting importance reflects a compromise: sensors that dominated under one condition (s0 for FD001, s11 for FD002) bot retain importance, but neither dominates as strongly as it did alone. PFI still produces an isolated spike on s8 (~ 0.36), unsupported by any other method. LIME remains heavily concentrated on s11 and s15, allocating aaround 0.35 and 0.40 respectively.

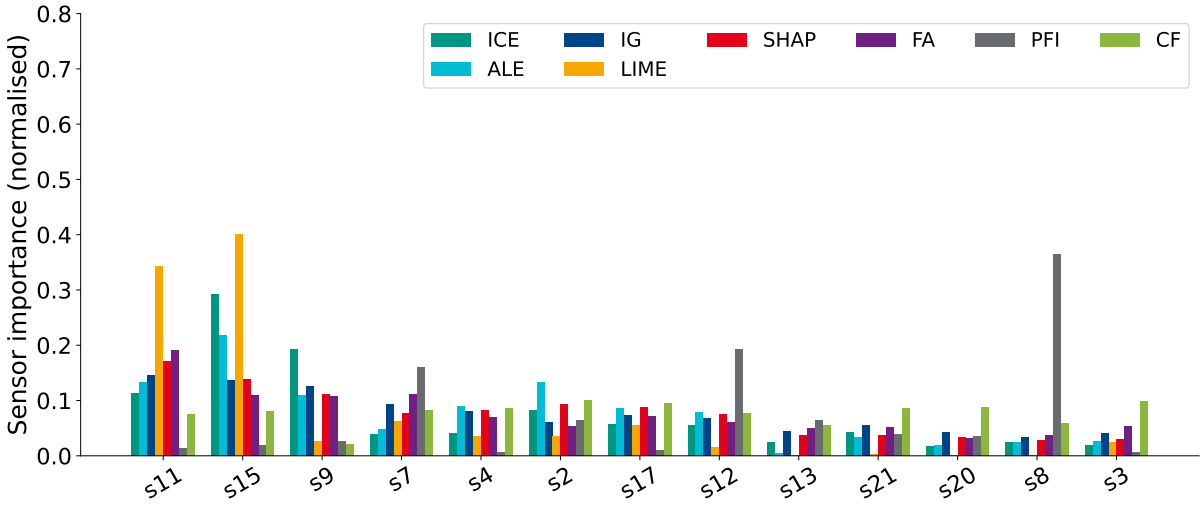

Figure 5.4: L1-normalised sensor importance across all eight methods for the pairwise-FD001-FD002 model. Sensors are ordered by decreasing mean importance.

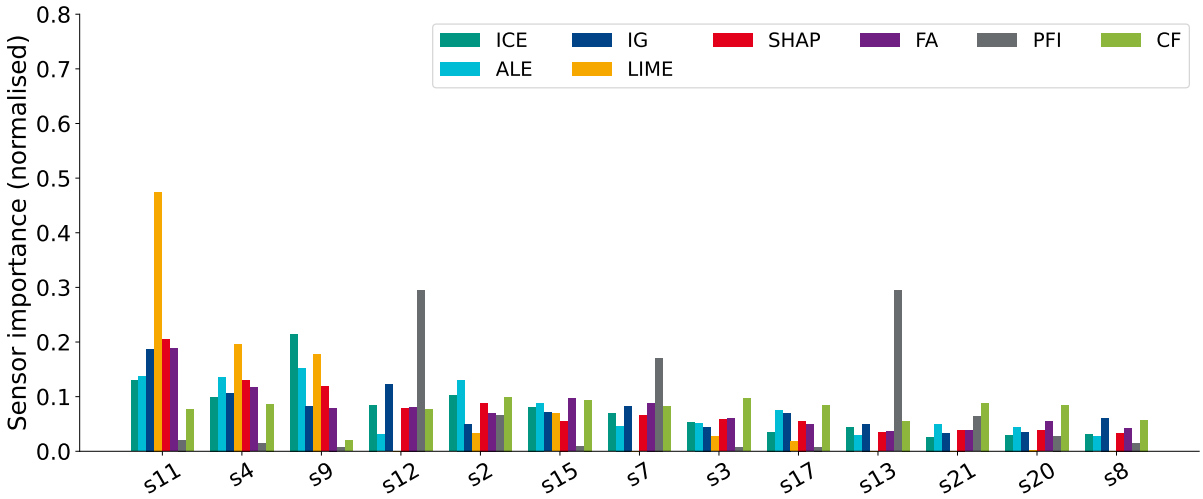

Figure 5.5: L1-normalised sensor importance across all eight methods for the model trained on all subsets. Sensors are ordered by decreasing mean importance.

The all-dataset model shows the most agreement across methods. Sesnor s11 is the clear front-runner, IG, SHAP, FA, ALE and ICE all. plcae it first, with normalised importance between 0.10 and 0.19. LIME assigns it roughly 0.48, continuing its pattern of over concentration. Sensors s4 and s9 follow in the secont tier, with s12, s2 and s15 in the middle ground. PFI deviates again, spiking on s12 (~ 0.30) and s13 (~ 0.30) while giving s11 minimal importance. CF distributes importance more evenly than in single-dataset cases but still does not match the consensus ranking.

5.4 Patterns Across Configuration

Several patterns hold across all four configuration.

The gradient-based and Shapley-based methods, IG, SHAP and FA, tend to cluster together. they rearely disagree on which sensor is most important ant their normalised magnitudes are usually within a few percentage points of each other. ALE often falls close to this cluster as well. These four methods form what might be called the consensus core of the comparison.

LIME consistently over-concentrates importance onto one or two sensors. It identifies the same top sensor as the consensus core in most cases, but assigns it two to three times the normalised weight. This is a known property of LIME's local linear surrogate fitting: when one feature dominates the local prediction surface, the surrogate can allocate neraly all explanatory weight to it, compressing the contributions of secondary features.

PFI produces isolated spikes on sensors that other methods rank as moderate or low importance. The specific sensor varies by configuration, s8 on FD002, s13 on the all-data model, but the pattern is consistent. These spikes likely reflect individual engines or narrow operating regimes where permuting that sensor causes disproportionate error, inflating the global average.

CF does not produce stable or interpretable rankings in this setting. Its results vary substantially across configurations and rarely agree with the consensus of the other methods. As discurssed in the verification section, counterfactual search is sensitive to the local geometry of the prediction surface and to the permitted perturbation ranges. On a 1D CNN with 13 sensor channels and complex temporal interactions, the search space appears large enough that the results are dominated by search dynamics rathen than meaningful feature effects.

The transisiton from single-dataset to multi-dataset training reveals a shift in which sensor dominate. On FD001alone, s9 lead. On FD002 and multi-dataset configurations, s11 takes over. This shift is discussed further in the context of the underlying physical degradation mechanism in Section 5.7.

global comparison

sensor comparison

time comparison

5.5 Feature Importance Results per Method

per method analysis

5.6 Comparison

correlation matrices feature importance of all methods, summed together -> heat maps

time component

5.7 Discussion

6 Conclusion and Outlook

6.1 Summary of Contributions

6.2 Answering the Research Questions

6.3 Outlook

Bibliography

Aas, Kjersti, Jullum, Martin and Løland, Anders (2021): *Explaining individual predictions when features are dependent: More accurate approximations to Shapley values*. Artificial Intelligence, Bd. 298, S. 103502. ISSN 00043702.

Alomari, Yazan, Andó, Mátyás and Baptista, Marcia L. (2023): *Advancing Aircraft Engine RUL Predictions: An Interpretable Integrated Approach of Feature Engineering and Aggregated Feature Importance*. Scientific Reports, Bd. 13, S. 13466.

Apley, Daniel W. and Zhu, Jingyu (2019): *Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models*.

Baptista, Marcia L., Goebel, Kai and Henriques, Elsa M. P. (2022): *Relation Between Prognostics Predictor Evaluation Metrics and Local Interpretability SHAP Values*. Artificial Intelligence, Bd. 306, S. 103667.

Baptista, Marcia L., Mishra, Madhav, Henriques, Elsa and Prendinger, Helmut (2024): *Using Explainable Artificial Intelligence to Interpret Remaining Useful Life Estimation with Gated Recurrent Unit*. In: *Annual Conference of the PHM Society*, Bd. 16.

Breiman, Leo (2001): *Random Forests*. Mach. Learn., Bd. 45, S. 5–32. ISSN 0885-6125.

Fisher, Aaron, Rudin, Cynthia and Dominici, Francesca (2019): *All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously*.

Gawde, Sujay, Patil, Shruti, Kumar, Satish, Kamat, Pooja, Kotecha, Ketan and Alfarhood, Sultan (2024): *Explainable Predictive Maintenance of Rotating Machines Using LIME, SHAP, PDP, ICE*. IEEE Access, Bd. 12, S. 29345–29361.

Goldstein, Alex, Kapelner, Adam, Bleich, Justin and Pitkin, Emil (2015): *Peeking Inside the Black Box: Visualizing Statistical Learning With Plots of Individual Conditional Expectation*. Journal of Computational and Graphical Statistics, Bd. 24, S. 44–65. ISSN 1061-8600, 1537-2715.

Heimes, Felix O. (2008): *Recurrent Neural Networks for Remaining Useful Life Estimation*. In: *2008 International Conference on Prognostics and Health Management*, S. 1–6. IEEE, Denver, CO, USA.

Hong, Chang Woo, Lee, Changmin, Lee, Kwangsuk, Ko, Min-Seung, Kim, Dae Eun and Hur, Kyeon (2020): *Remaining Useful Life Prognosis for Turbofan Engine Using Explainable Deep Neural Networks with Dimensionality Reduction*. Sensors, Bd. 20, S. 6626.

Kusner, Matt J., Loftus, Joshua R., Russell, Chris and Silva, Ricardo (2018): *Counterfactual Fairness*.

Li, Hao, Wang, Zhuojian and Li, Zhenglin (2022): *An Enhanced CNN-LSTM Remaining Useful Life Prediction Model for Aircraft Engine with Attention Mechanism*. PeerJ Computer Science, Bd. 8, S. e1084.

Li, Xiang, Ding, Qian and Sun, Jian-Qiao (2018): *Remaining Useful Life Estimation in Prognostics Using Deep Convolution Neural Networks*. Reliability Engineering & System Safety, Bd. 172, S. 1–11.

Lundberg, Scott and Lee, Su-In (2017a): *A Unified Approach to Interpreting Model Predictions*.

Lundberg, Scott M. and Lee, Su-In (2017b): *A unified approach to interpreting model predictions*. In: *Advances in Neural Information Processing Systems*, Bd. 30.

Molnar, Christoph (2022): *Interpretable Machine Learning*. Leanpub. Available online.

Ramasso, Emmanuel and Saxena, Abhinav (2014): *Performance Benchmarking and Analysis of Prognostic Methods for CMAPSS Datasets*. International Journal of Prognostics and Health Management, Bd. 5, S. 1–15.

Ribeiro, Marco Tulio, Singh, Sameer and Guestrin, Carlos (2016): *Model-Agnostic Interpretability of Machine Learning*.

Rudin, Cynthia (2019): *Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead*. Nature Machine Intelligence, Bd. 1, S. 206–215.

Sateesh Babu, Giduthuri, Zhao, Peilin and Li, Xiao-Li (2016): *Deep Convolutional Neural Network Based Regression Approach for Estimation of Remaining Useful Life*. In: *Database Systems for Advanced Applications (DASFAA 2016)*, Bd. 9642 von *Lecture Notes in Computer Science*, S. 214–228. Springer, Cham.

Saxena, Abhinav, Goebel, Kai, Simon, Don and Eklund, Neil (2008): *Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation*. In: *2008 International Conference on Prognostics and Health Management*, S. 1–9. IEEE, Denver, CO, USA.

Serradilla, Oscar, Zugasti, Ekhi, Cernuda, Cristobal, Aranburu, Aitor, de Okariz, Jaime Ramirez and Zurutuza, Urko (2020): *Interpreting Remaining Useful Life Estimations Combining Explainable Artificial Intelligence and Domain Knowledge in Industrial Machinery*. In: *2020 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*, S. 1–8. IEEE.

Shapley, L. S. (1953): *Stochastic Games**. Proceedings of the National Academy of Sciences, Bd. 39, S. 1095–1100.

Sundararajan, Mukund, Taly, Ankur and Yan, Qiqi (2017): *Axiomatic Attribution for Deep Networks*.

Verma, Sahil, Boonsanong, Varich, Hoang, Minh, Hines, Keegan E., Dickerson, John P. and Shah, Chirag (2022): *Counterfactual Explanations and Algorithmic Recourses for Machine Learning: A Review*.

Wachter, Sandra, Mittelstadt, Brent and Russell, Chris (2018): *Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR*.

Wang, Tianyi, Yu, Jianbo, Siegel, David and Lee, Jay (2008): *A Similarity-Based Prognostics Approach for Remaining Useful Life Estimation of Engineered Systems*. In: *2008 International Conference on Prognostics and Health Management*, S. 1–6. IEEE, Denver, CO, USA.

Wu, Yuting, Yuan, Mei, Dong, Shaopeng, Lin, Li and Liu, Yongqian (2018): *Remaining Useful Life Estimation of Engineered Systems Using Vanilla LSTM Neural Networks*. Neurocomputing, Bd. 275, S. 167–179.

Zeiler, Matthew D. and Fergus, Rob (2014): *Visualizing and Understanding Convolutional Networks*. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, S. 818–833. Springer.

Zhao, Rui, Yan, Ruqiang, Wang, Jinjiang and Mao, Kezhi (2017): *Learning to Monitor Machine Health with Convolutional Bi-Directional LSTM Networks*. Sensors, Bd. 17, S. 273.

Zheng, Shuai, Ristovski, Kosta, Farahat, Ahmed and Gupta, Chetan (2017): *Long Short-Term Memory Network for Remaining Useful Life Estimation*. In: *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)*, S. 88–95. IEEE, Dallas, TX, USA.

Štrumbelj, Erik and Kononenko, Igor (2014): *Explaining prediction models and individual predictions with feature contributions*. Knowledge and Information Systems, Bd. 41, S. 647–665. ISSN 0219-1377, 0219-3116.

Appendix

A.1 Individual Results

A.2 Comparative Results